

21720 Programació concurrent

4. Solucions per *hardware*

- Ineficiència dels algorismes que solucionen el problema d'exclusió mútua
 - Mida dels *arrays*: Coherència de caché
 - Problema de l'intercalat: Resultat imprevisible
- Solució: **Inhabilitació de les interrupcions**
 - No és acceptable per a processos d'usuari
 - S'ha de fer per a tots els processadors si no el problema de l'intercalat persisteix

Sincronització per *hardware*

- **Instruccions** que llegeixen i modifiquen els valors de registre sense ser interrompudes, asseguren coherència de caché i introdueixen barreres de memòria
- **Registres** com a objectes compartits amb mètodes de lectura i escriptura. Tipus:
 - **Segurs**: De consistència dèbil. Retornen el darrer valor escrit si no hi ha operacions de L/E concurrents
 - **Regulars**: Consistència per inactivitat. Retornen el mateix valor després d'un període d'inactivitat entre el darrer *write* i el següent *get*
 - **Atòmics**: Retornen el darrer valor escrit (*get*), si hi ha varis *write* concurrents en retornaran un d'ells

Primitives de *hardware*

- **Registres RMW** (*Read-Modify-Write*): Primitives o instruccions atòmiques expressades com a abstraccions
- Tipus d'instruccions que implementen
 - ***get&set***: assigna un valor al registre i retorna l'anterior
 - ***get&add***: incrementa el valor del registre amb el valor indicat i retorna l'anterior
 - ***test&set***: verifica el valor del registre, si és 0 li posa 1, retorna l'assignat
 - ***swap***: intercanvia el valor de dos registres
 - ***compare&swap***: compara el valor del registre amb un altre, si són iguals assigna un nou valor i retorna l'anterior

Exclusió mútua amb *get&set*

```
mutex = 0

def lock():
    while getAndSet(mutex, 1) != 0:
        pass

def unlock():
    mutex = 0
```

- La variable *mutex* val 0 si no hi cap procés a la SC, tot el temps que valgui 1 estarà dins el *while* bloquejant
- Implementació amb C: `__atomic_exchange_n`

https://gcc.gnu.org/onlinedocs/gcc/_005f_005fatomic-Builtins.html

- Implementació amb Java: mètode `getAndSet` de la classe `AtomicReference`

<http://docs.oracle.com/javase/7/docs/api/java/util/concurrent/atomic/AtomicReference.html>

counter_get_and_set.c

```
int mutex = 0; // 0 vol dir que no hi ha processos a la SC
int counter = 0;

void lock() {
    //getAndSet(mutex,1,AsseguraOrdreSeq)
    while(__atomic_exchange_n(&mutex, 1, __ATOMIC_SEQ_CST));
}

void unlock() {
    __atomic_thread_fence(__ATOMIC_RELEASE);
    mutex = 0;
}
```

CounterGetAndSet.java

```
static final int THREADS = 4;
static final int MAX_COUNT = 1000000;
static AtomicReference<Integer> mutex;
static volatile int n = 0;
int id;

public CounterGetAndSet(int id) {
    this.id = id;
}

@Override
public void run() {
    int fi = MAX_COUNT / THREADS;
    System.out.println("Thread " + id);
    for (int i = 0; i < fi; i++) {
        // Get&Set
        while (mutex.getAndSet(1) != 0) {
        }
        n += 1; //SC
        mutex.set(0);
    }
}

public static void main(String[] args) throws InterruptedException {
    Thread[] threads = new Thread[THREADS];
    mutex = new AtomicReference<Integer>(0);
```

Exclusió mútua amb *get&add*

- Similar a l'algorisme del forn: cada procés té un nombre i espera el seu torn
- L'obtenció del següent nombre és atòmica i no es necessita l'array

```
number = 0
turn = 0

def lock():
    """ current is a local variable """
    current = getAndAdd(number, 1)
    while current != turn:
        pass

def unlock():
    getAndAdd(turn, 1)
```

- Implementació amb C: `__atomic_fetch_add`
- Implementació amb Go: `atomic.AddUint32`

<https://golang.org/pkg/sync/atomic/#AddUint32>

- Implementació amb Java: mètode `GetAndAdd` de la classe `AtomicInteger`

counter_get_and_add.c

```
int number = 0;
int turn = 0;
int counter = 0;

void lock() {
    int current;
    current = __atomic_fetch_add(&number, 1, __ATOMIC_RELAXED);
    while(current != turn);
}

void unlock() {
    __atomic_fetch_add(&turn, 1, __ATOMIC_RELEASE);
}

void *count(void *ptr) {
    long i, max = MAX_COUNT/NUM_THREADS;
    int tid = ((struct tdata *) ptr)->tid;
    for (i=0; i < max; i++) {
        lock();
        printf("Counter %d Fil %d\n", counter, tid );
        counter += 1;
        unlock();
    }
    printf("End %d counter: %d\n", tid, counter);
}
```

gocounter_get_and_add.go

```
var (  
    counter int  
    number  uint32  
    turn    uint32 = 1 // AddUint32 returns the new value  
)  
  
func lock() {  
    var current uint32  
    current = atomic.AddUint32(&number, 1)  
    /* We have to use LoadUint32 because there is no volatile vars  
       and turn is read from the local cache */  
    for current != atomic.LoadUint32(&turn) {  
    }  
}  
  
func unlock() {  
    atomic.AddUint32(&turn, 1)  
}  
  
func run(id, counts int, done chan bool) {  
    for i := 0; i < counts; i++ {  
        lock()  
        counter++  
        unlock()  
    }  
    fmt.Printf("End %d counter: %d\n", id, counter)  
    done <- true  
}
```

Exclusió mútua amb *test&set*

```
mutex = 0

def lock():
    while TAS(mutex) == 1:
        pass

def unlock():
    mutex = 0
```

- Té un únic operant si val 0 li assigna un 1 i retorna 0 si no retorna un 1
- Implementació amb C: `__atomic_test_and_set`
- Implementació amb Java: mètode `compareAndSet` de la classe `AtomicBoolean`

counter_test_and_set.c

```
char mutex = 0;
int counter = 0;

void lock() {
    while(__atomic_test_and_set(&mutex, __ATOMIC_SEQ_CST));
}

void unlock() {
    __atomic_thread_fence(__ATOMIC_RELEASE);
    mutex = 0;
}

void *count(void *ptr) {
    long i, max = MAX_COUNT/NUM_THREADS;
    int tid = ((struct tdata *) ptr)->tid;
    for (i=0; i < max; i++) {
        lock();
        counter += 1;
        unlock();
    }
    printf("End %d counter: %d\n", tid, counter);
}
```

Exclusió mútua amb *swap*

```
mutex = 0

def lock():
    local = 1
    while local != 0:
        swap(mutex, local)

def unlock():
    mutex = 0
```

- Fa l'intercanvi atòmic de dues posicions de memòria. Igual que el TAS, la diferència és que el valor anterior de *mutex* se verifica en la variable local usada per a l'intercanvi
- Implementació amb C: `__atomic_exchange`
- Implementació amb GO: `atomic.SwapInt32`

counter_swap.c

```
int mutex = 0;
int counter = 0;

void lock(int i) {
    int local = 1;
    while (local) {
        //swap(origen, final, return)
        __atomic_exchange(&mutex, &local, &local, __ATOMIC_SEQ_CST);
    }
}

void unlock(int i) {
    __atomic_thread_fence(__ATOMIC_RELEASE);
    mutex = 0;
}
```

gocounter_swap.go

```
const (  
    Procs      = 4  
    MaxCount   = 10000000  
    Goroutines = 4  
)  
  
var (  
    counter int  
    mutex   int32  
)  
  
func lock() {  
    for atomic.SwapInt32(&mutex, 1) == 1 {  
        // While it returns 1  
    }  
}  
  
func unlock() {  
    mutex = 0  
}  
  
func run(id, counts int, done chan bool) {  
    for i := 0; i < counts; i++ {  
        lock()  
        counter++  
        unlock()  
    }  
    fmt.Printf("End %d counter: %d\n", id, counter)  
    done <- true  
}
```

Exclusió mútua amb *compare&swap*

```
mutex = 0

def lock():
    local = 0
    while not CAS(mutex, local, 1):
        local = 0

def unlock():
    mutex = 0
```

- Tres operands: **Registre** (conté la posició de memòria que es compararà), **valor esperat** (si coincideix amb el del registre s'assignarà el nou valor) i **nou valor**, retorna el valor anterior
- Implementació amb C: `__atomic_compare_exchange_n`
- Implementació amb Go: `atomic.CompareAndSwapInt32`
- Implementació amb Java: mètode `compareAndSet` de la classe `AtomicInteger`

counter_compare_and_swap.c

```
int mutex = 0;
int counter = 0;

void lock(int i) {
    int local;
    do {
        local = 0;
    } while (! __atomic_compare_exchange_n(&mutex, &local, 1, 0, __ATOMIC_RELAXED, __ATOMIC_RELAXED));
}

void unlock(int i) {
    __atomic_thread_fence(__ATOMIC_RELEASE);
    mutex = 0;
}
```

CounterCompareAndSwap.java

```
static final int THREADS = 4;
static final int MAX_COUNT = 1000000;
static AtomicInteger mutex = new AtomicInteger(0);
static volatile int n = 0;
int id;

public CounterCompareAndSwap(int id) {
    this.id = id;
}

@Override
public void run() {
    int fi = MAX_COUNT / THREADS;
    System.out.println("Thread " + id);
    for (int i = 0; i < fi; i++) {
        // Compare&Swap
        while (!mutex.compareAndSet(0, 1)) {
            Thread.yield();
        }
        n += 1; //SC
        mutex.set(0);
    }
}
```

- No hi ha instruccions de hardware unificades per a totes les architectures
- No hi ha estandardització a nivell de llenguatges de programació
- Les solucions són aplicables a n processadors
- Simples i fàcils de verificar
- Es poden definir diferents regions crítiques cada una amb la seva variable
- Inconvenient d'usar espera activa: consum de CPU
- La selecció és arbitrària i això pot produir inanició: Espera infinita

- A tots els exemples anteriors la regió crítica consisteix només en incrementar un comptador compartit i exemplifica de manera molt simple el problema de la concurrència
- Per solucionar el problema real no cal un *spinlock* basta usar *get&add*

```
int counter = 0;

void *count(void *ptr) {
    long i, max = MAX_COUNT/NUM_THREADS;
    int tid = ((struct tdata *) ptr)->tid;
    int c;
    for (i=0; i < max; i++) {
        c = __atomic_add_fetch(&counter, 1, __ATOMIC_SEQ_CST);
    }
    printf("End %d counter: %d\n", tid, c);
}
```

```
public void run() {
    int fi = MAX_COUNT / THREADS;
    System.out.println("Thread " + id);
    for (int i = 0; i < fi; i++) {
        n.getAndIncrement();
    }
}
```

```
var (
    counter int32
)

func run(id, counts int, done chan bool) {
    var c int32
    for i := 0; i < counts; i++ {
        c = atomic.AddInt32(&counter, 1)
    }
    fmt.Printf("End %d counter: %d\n", id, c)
    done <- true
}
```

counter_ultimate.c
gocounter_ultimate.go
CounterUltimate.java